

Sistema de probabilidad usado en `zanni_pytorch.py`

1 Resumen

El script `zanni_pytorch.py` calcula una estrategia optima de ofertas para el problema “Zanni de Oro”. Aunque el nombre del archivo menciona `pytorch`, la version actual no usa PyTorch ni librerias externas: usa un modelo probabilistico continuo, condiciones de primer orden y busqueda binaria.

El intervalo de precios usado por el programa es

$$[L, H] = [620, 880],$$

por lo que su largo es

$$S = H - L = 260.$$

El codigo tambien considera

$$N = 10000$$

monedas.

2 Normalizacion

Para trabajar con formulas mas simples, cada oferta real b se transforma a una oferta normalizada t en el intervalo $[0, 1]$:

$$t = \frac{b - L}{S}.$$

Equivalentemente, una oferta normalizada t corresponde al precio real

$$b = L + St.$$

Asi, $t = 0$ representa 620, y $t = 1$ representa 880.

3 Distribucion probabilistica

El programa modela el precio de reserva normalizado R con funcion de distribucion acumulada

$$F(t) = t^2, \quad 0 \leq t \leq 1.$$

Por lo tanto,

$$\Pr(a < R \leq b) = F(b) - F(a) = b^2 - a^2.$$

La densidad asociada es

$$f(t) = F'(t) = 2t.$$

Esto significa que los valores altos de reserva son mas probables que los bajos, porque la densidad crece linealmente con t .

4 Ganancia esperada

Supongamos que se hacen k ofertas normalizadas

$$0 < t_1 < t_2 < \dots < t_k \leq 1.$$

Definimos $t_0 = 0$. Si la reserva cae en el intervalo

$$t_{i-1} < R \leq t_i,$$

entonces la oferta que se acepta es t_i . La ganancia normalizada de esa oferta es

$$1 - t_i,$$

y la probabilidad de que ocurra ese caso es

$$\Pr(t_{i-1} < R \leq t_i) = t_i^2 - t_{i-1}^2.$$

Por lo tanto, la ganancia esperada normalizada es

$$G(t_1, \dots, t_k) = \sum_{i=1}^k (1 - t_i)(t_i^2 - t_{i-1}^2).$$

El programa implementa exactamente esta expresion en la funcion

`expected_profit_normalized.`

5 Optimizacion

Para maximizar G , el codigo usa las condiciones estacionarias. Para una oferta interna t_i , con $1 \leq i < k$, la condicion de primer orden lleva a

$$t_{i-1}^2 - 3t_i^2 + 2t_it_{i+1} = 0.$$

Despejando t_{i+1} , se obtiene la recurrencia usada por el programa:

$$t_{i+1} = \frac{3t_i^2 - t_{i-1}^2}{2t_i}.$$

La ultima oferta t_k satisface la condicion terminal

$$t_{k-1}^2 + 2t_k - 3t_k^2 = 0.$$

Para el caso $k = 1$, esta condicion queda como

$$2t_1 - 3t_1^2 = 0,$$

de donde el optimo no trivial es

$$t_1 = \frac{2}{3}.$$

El algoritmo elige t_1 mediante busqueda binaria en $[0, 1]$, genera las siguientes ofertas con la recurrencia anterior y ajusta t_1 hasta cumplir la condicion terminal.

6 Conversion a ganancias reales

Una vez obtenida la ganancia esperada normalizada G , el programa calcula la ganancia esperada por moneda como

$$\text{ganancia por moneda} = S \cdot G.$$

Como hay $N = 10000$ monedas, la ganancia esperada total es

$$\text{ganancia total} = N \cdot S \cdot G.$$

7 Resultados del script

Ejecutando

```
python3 zanni_pytorch.py --max-bids 5
```

se obtienen los siguientes valores:

Fase	Ofertas optimas	Ganancia por moneda	Ganancia total
1	793.333333	38.518519	385185.19
2	755.652174, 823.478261	53.081285	530812.85
3	733.634204, 790.451306, 837.798892	60.815843	608158.43
4	718.886355, 768.329533, 809.532181, 846.256281	65.630647	656306.47
5	708.184178, 752.276267, 789.019674, 821.769233, 851.860970	68.922448	689224.48

8 Pseudocodigo

entrada: k numero de ofertas

salida: $t_1, \dots, t_k$ ofertas normalizadas optimas

1. Buscar t_1 en el intervalo $[0, 1]$.

2. Fijar $t_0 = 0$.

3. Para $i = 1, \dots, k - 1$, calcular

$$t_{i+1} = \frac{3t_i^2 - t_{i-1}^2}{2t_i}.$$

4. Evaluar la condicion terminal

$$t_{k-1}^2 + 2t_k - 3t_k^2.$$

5. Ajustar t_1 por busqueda binaria hasta que la condicion terminal sea cercana a cero.

6. Convertir cada t_i a precio real con

$$b_i = 620 + 260t_i.$$

9 Conclusion

El sistema probabilístico usado por el programa se basa en una reserva aleatoria continua con distribución $F(t) = t^2$. La estrategia óptima no se obtiene por simulación, sino maximizando la ganancia esperada mediante condiciones analíticas. La única parte numérica es la búsqueda binaria del primer punto t_1 , que permite construir todas las demás ofertas.